

Unidad 2

Programación de hilos

2.3 Sincronización

Programación de servicios y procesos

Curso 2025/2026

Contenido

- ▶ Secciones críticas: synchronized
- ▶ Productores y consumidores: wait y release

Secciones críticas - synchronized

Programación de hilos

Condición de carrera

- ▶ Ocurre cuando:
 - ▶ Dos o más hilos o procesos acceden a un recurso compartido
 - ▶ Al menos uno lo escribe o modifica
 - ▶ El orden en que los hilos acceden al recurso **cambia el resultado final**
- ▶ Se llama "condición de carrera" porque depende de quien llega antes
- ▶ Ejemplos de recursos compartidos en los que pueden producirse condiciones de carrera:
 - ▶ Una variable que cuenta visitas
 - ▶ Una lista en la que varios hilos añaden elementos
 - ▶ Una cuenta bancaria

Sección crítica

- ▶ La solución a la condición de carrera es la definición y la protección de la sección crítica
- ▶ Sección crítica: conjunto de instrucciones que acceden a un recurso compartido y deben ejecutarse bajo **exclusión mutua**
- ▶ En otras palabras, solo se deben ejecutar por un hilo a la vez

Sección crítica fragmentada

- ▶ La sección crítica puede estar fragmentada, es decir, repartida en varios métodos o bloques de código
- ▶ La condición de carrera no depende de que dos hilos ejecuten el mismo código a la vez, sino de que **accedan al mismo recurso**
- ▶ Por tanto, todo código que accede al recurso debe ser considerado una sección crítica

Ejemplo: Contador

- ▶ Tenemos la clase contador con métodos para incrementar y decrementar
- ▶ ¿Qué pasa si creamos dos hilos, uno usa increment() y otro usa decrement()?

```
public class Counter {  
    int value;  
  
    public Counter(int value) {  
        this.value = value;  
    }  
  
    public void increment() {  
        value++;  
    }  
  
    public void decrement() {  
        value--;  
    }  
  
    public int getValue() {  
        return value;  
    }  
}
```

Ejemplo: contador

```
public static void main(String[] args) {
    Counter c = new Counter(100);
    Thread tinc = new Thread(() -> {
        for (int i = 0; i < 100; i++) {
            c.increment();
            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
            }
        }
        System.out.println("Finishing inc. Final value = " + c.getValue());
    });
    Thread tdec = new Thread(() -> {
        for (int i = 0; i < 100; i++) {
            c.decrement();
            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
            }
        }
        System.out.println("Finishing dec. Final value = " + c.getValue());
    });
    tinc.start();
    tdec.start();
}
```


Ejemplo: contador

- ▶ Pregunta: ¿el resultado es 100?

¿Por qué ocurre esto?

- ▶ Las operaciones ++ y -- no son atómicas
- ▶ Son lecturas y escrituras
- ▶ Puede colarse un decremento entre la lectura y escritura de un incremento
 - ▶ o viceversa
- ▶ La operación intermedia se pierde

Solución

- ▶ Palabra clave synchronized
- ▶ Controla el acceso a métodos, convirtiéndolos en secciones críticas

```
public class Counter {  
    int value;  
  
    public Counter(int value) {  
        this.value = value;  
    }  
  
    public synchronized void increment() {  
        value++;  
    }  
  
    public void synchronized decrement() {  
        value--;  
    }  
  
    public int getValue() {  
        return value;  
    }  
}
```

Métodos sincronizados

- ▶ La palabra clave `synchronized` aplicada a métodos de una clase impide que se puedan ejecutar a la vez sobre el mismo objeto - métodos sincronizados
- ▶ Cuidado: sobre **objetos distintos** (de la misma clase) sí que se pueden ejecutar a la vez
- ▶ Manera de impedirlo: métodos estáticos (`static synchronized`)

Ejercicio propuesto

- ▶ Actividad U2. AP3 - ejercicio 1 (cuenta del banco)

Bloques sincronizados

- ▶ `synchronized(objeto)`
 - ▶ A menudo el objeto es `this` (el objeto que contiene el bloque)
 - ▶ También se puede crear un objeto para ello de clase `Object`
- ▶ Para sincronizar bloques entre sí, deben referenciar al mismo objeto - llamado "monitor"

```
public void myMethod() {  
    int someValue;  
    ...  
    synchronized(this) {  
        someValue++;  
        System.out.println("Value changed: " + someValue);  
    }  
    ...  
}
```

Ejercicio propuesto

- ▶ Actividad U2. AP3 - ejercicio 2 (cuenta del banco)

Mecanismo `synchronized` en otros lenguajes

- ▶ Lenguajes con mecanismos similares:
 - ▶ C# (palabra clave `lock`)
 - ▶ Kotlin (palabra clave `synchronized`)
 - ▶ Python (with `lock`, no nativo)
- ▶ Lenguajes con mecanismos distintos:
 - ▶ C/C++, Go, Rust
 - ▶ Utilizan `mutex` (bloqueo "a mano" antes de sección crítica y desbloqueo después)
 - ▶ Más proclives a problemas (aunque Rust intenta evitarlos por compilación)
 - ▶ Al tener más control permiten casos de uso avanzados

Productores y consumidores: wait y release

Programación de hilos

Problema productor-consumidor

- ▶ Problema habitual programación concurrente
 - ▶ Buffer de datos
 - ▶ Hilos productores: introducen datos
 - ▶ Hilos consumidores: extraen datos
- ▶ Condiciones:
 - ▶ Si está lleno, los hilos productores deben esperar a que el buffer tenga sitio
 - ▶ Si está vacío, los hilos consumidores deben esperar a que el buffer tenga datos

Problema productor-consumidor

- ▶ Queremos evitar:
 - ▶ Un productor insertando mientras otro también inserta → inconsistencia
 - ▶ Un consumidor leyendo mientras el buffer está vacío → error
 - ▶ Productores llenando el buffer sin freno → overflow
- ▶ ¿Es suficiente con usar la palabra clave synchronized? → NO
 - ▶ Soluciona el problema del acceso simultáneo, PERO:
 - ▶ Si el buffer está lleno, el productor no puede seguir, pero tampoco debe bloquear el monitor indefinidamente.
 - ▶ Si el buffer está vacío, el consumidor no puede seguir, pero tampoco debe bloquear el monitor eternamente

Métodos de espera y notificación

- ▶ Métodos `wait()`, `notify()` y `notifyAll()`
- ▶ Definidos en la clase `java.lang.Object`
 - ▶ Todos los objetos los tienen
- ▶ Solucionan el problema productor/consumidor

Método wait()

- ▶ Se puede llamar dentro de un bloque sincronizado
- ▶ Al ser llamado, la JVM:
 1. Pone el hilo en reposo
 2. Libera el objeto bloqueado por el bloque sincronizado
 - ▶ Otros hilos en espera pueden continuar

Métodos notify() y notifyAll()

- ▶ Son llamados por un hilo que ha terminado su tarea dentro de una sección crítica antes de salir de ella
- ▶ Indican a la JVM que puede despertar a un hilo que ha hecho wait()
- ▶ Diferencias:
 - ▶ Con notify(), la JVM elige uno de los hilos en espera y lo despierta
 - ▶ Con notifyAll(), la JVM despierta a todos los hilos en espera

Ejemplo - buffer de un entero

```
public class SharedData {  
    int data;  
  
    public int get() {  
        return data;  
    }  
  
    public void put(int newData) {  
        data = newData;  
    }  
}
```

Producer

```
public class Producer extends Thread {
    SharedData data;

    public Producer(SharedData data) {
        this.data = data;
    }

    @Override
    public void run() {
        for (int i = 0; i < 50; i++) {
            data.put(i);
            System.out.println("Produced number " + i);
            try {
                Thread.sleep(10);
            } catch (Exception e) {}
        }
    }
}
```


Consumidor

```
public class Consumer extends Thread {
    SharedData data;

    public Consumer(SharedData data) {
        this.data = data;
    }

    @Override
    public void run() {
        for (int i = 0; i < 50; i++) {
            int n = data.get();
            System.out.println("Consumed number " + n);
            try {
                Thread.sleep(10);
            } catch (Exception e) {}
        }
    }
}
```

Función principal

- Crea un objeto SharedData, un hilo de cada tipo y los pone a correr

```
public static void main(String[] args) {  
    SharedData sd = new SharedData();  
    Producer p = new Producer(sd);  
    Consumer c = new Consumer(sd);  
    p.start();  
    c.start();  
}
```

Resultado de ejecución

► ¿Qué está pasando?

```
Consumed number 0
Produced number 0
Consumed number 0
Produced number 1
Consumed number 1
Produced number 2
Produced number 3
Consumed number 3
Produced number 4
Consumed number 4
Consumed number 4
```

Sincronizamos métodos en el buffer...

```
public class SharedData {  
    int data;  
  
    public synchronized int get() {  
        return data;  
    }  
  
    public synchronized void put(int newData) {  
        data = newData;  
    }  
}
```

¿Solucionamos el problema?

► ¿Por qué ocurre esto?

```
Consumed number 0
Produced number 0
Consumed number 1
Produced number 1
Produced number 2
Consumed number 1
Produced number 3
Consumed number 3
Produced number 4
Consumed number 4
```

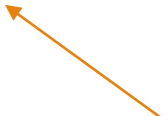
Solución - creamos un flag

```
public class SharedData {  
    int data;  
    boolean available = false;  
  
    public synchronized int get() {  
        available = false;  
        return data;  
    }  
  
    public synchronized void put(int newData) {  
        data = newData;  
        available = true;  
    }  
}
```

Y además...

```
public class SharedData {  
    int data;  
    boolean available = false;  
  
    public synchronized int get() {  
        if (!available)  
            try {  
                wait();  
            } catch (Exception e) {  
            }  
        available = false;  
        notify();  
        return data;  
    }  
}
```

Ojo a esto



```
public synchronized void put(int newData) {  
    if (available)  
        try {  
            wait();  
        } catch (Exception e) {  
        }  
    data = newData;  
    available = true;  
    notify();  
}
```

Mecanismo wait/release en otros lenguajes

- ▶ Métodos parecidos a wait() y release() prácticamente existen en todos los lenguajes, si bien no están integrados en los objetos por defecto
 - ▶ Kotlin: igual que Java
 - ▶ C/C++: pthreads
 - ▶ C#: Clase Monitor
 - ▶ Python: Clase Condition
 - ▶ Go: clase Cond, aunque es más común usar otro mecanismo para comunicación entre hilos: channels
 - ▶ Rust: clase Condvar

Ejercicio propuesto

- ▶ Actividad U2. AP4 (lavaplatos)